

How Gaffer's Clipboard Rates Players

Executive Summary

The quantification of a football performance into a single number from 0.0 to 10.0 has long been a complex and controversial process. At its core, it is an exercise in dimensionality reduction - distilling a web of complex and often interconnected statistics down to a single number. Industry-standard algorithms, such as those employed by platforms like *Sofascore* and *Fotmob*, have made significant strides in this area. However, fans frequently view them critically, as these systems carry inherent biases and operate as 'black boxes' with little transparency into how they actually weigh and analyse data.

The Gaffer's Clipboard Analytics Engine seeks to address these flaws by providing a transparent, mathematically rigorous 0.0 to 10.0 scale. While it has its own limitations (relying on statistics extracted directly from EA FC/FIFA screens), the engine operates completely offline. It feeds match telemetry through a strict pipeline involving data standardisation, principal component analysis, and contextual heuristics. By evaluating player performance solely on what unfolds on the virtual pitch, the engine offers users a clear, unbiased view of performance, fostering trust and confidence in tactical decisions and long-term tracking.

I. Introduction and Philosophy

In today's modern world, there is an increasing emphasis on viewing things through the lens of data and statistics. Whether assessing a company's performance over a financial quarter, a student's academic abilities in school, or the supposed success of a government, organisations and individuals increasingly seek the ability to strip away human error and bias to view performance through an objective lens. Moreover, the field of sports is no different. Whilst there are a few other similar fields, like politics, few others come close to the level of tribalism and, therefore, inherent biases found in sports and amongst their stakeholders. Along with how prevalent sports can be in our daily lives and the growing economic value they create, this explains why sports teams, fans, and others involved are increasingly focused on quantifying sports into objective data.

However, this is harder than may be initially thought, and harder with some sports than others. For example, analysing the performance of a sprinter is easy; there is only one real metric that matters - their time. Moreover, if one wants to go deeper, with cameras and basic timing

technology, it is relatively easy to track their top sprint speed, their acceleration over the first x meters, and so on. Team sports, on the other hand, are a different ball game, and football more than most. In the NBA, for example, it is somewhat easy to determine who did well in a game without watching it, simply by looking at simple metrics: points, assists, rebounds, steals, turnovers, etc. However, whilst football may have similar metrics: goals, assists, passes, tackles, shots, etc, these are nowhere near as informative. Whilst in basketball, fans may debate about who is the best ever by looking at career averages like points or field goals, the only statistic in football that comes somewhat close to that level of prominence in these debates is goals scored, and that is only one aspect of a player, and an even smaller aspect if that player is not a striker.

Part of why football is so much harder to quantify is that luck is a more significant factor in it than in other team sports. With such long matches and yet such small scorelines, it is common to see players and teams perform well, yet draw with or lose to teams supposedly performing much worse, simply because they had one moment of luck. Another is the sport's complexity and the expectations placed on players. Whilst a golfer may only be expected to put the ball towards the hole, a single footballer may be expected to consistently pass the ball to his teammates successfully (but if he is making safe passes too much that may be seen as bad), tackle opposing players and intercept their passes, cover a lot of ground across the pitch, create chances for their teammates, and get in good positions to shoot themselves. This is before one looks at aspects of their performance that are even harder to quantify in a single number. Take pressing: the action of a player or team applying pressure to the opponent whilst they have the ball, attempting to force turnovers, minimise time on the ball, and win possession. It is incredibly hard to scale how well a player or team performs at this into a single number.

Therefore, reducing all aspects of a player's performance, a matrix of spatial, temporal and event-based telemetry, into one unified player rating poses significant mathematical and epistemological challenges. The engine evaluates a variety of data types expressed in different ways - some unbounded, others not normally distributed - and attempting to quantify them on a finite scale.

At least amongst fans, a few apps and their commercial algorithms dominate this space (like *Sofascore*, *Fotmob* and *Whoscored*). These are increasingly significant in shaping how fans view players, influencing public perception and even professional scouting. Fans who have not watched a game will still often confidently form an opinion on how a player played based on a single number. However, at the same time, a lot of fans are noticing the serious deficiencies and drawbacks of these models, from their strong attacking bias (teams of the week and of tournaments generated by these services often do not represent a realistic team, usually packed too full of attacking players), to their lack of transparency (serious frustration comes from the black box nature of these algorithms, fans not being allowed to understand the inputs or how the algorithm works, only the output). The so-called "eye test" still is, and always will be,

important in evaluating a performance, and fans often express annoyance when mathematical ratings do not reflect what they have watched with their own eyes.

Performing the previously mentioned analyses, not on real football matches but on simulated ones via a video game, is even more complicated. EA FC's simulation and the stats it generates from it have their own drawbacks. Whilst the video game has its own match rating system built in, it is deeply flawed. The in-game algorithm evaluates players in a tactical vacuum, heavily over-indexing on raw goal involvements while routinely ignoring positional responsibilities. For example, a fullback explicitly instructed to stay back and defend will consistently receive artificially low match ratings for failing to register attacking output. Furthermore, the game's temporal scaling is highly volatile; an 85th-minute substitute can drastically inflate their match score with a mere handful of successful passes, breaking managerial immersion. These limitations are exacerbated by how the Gaffer's Clipboard application works - to make it easier to use, and the data logging process smoother and simpler, only 16 stats are collected directly from each player's performance, along with 16 stats collected on each team's collective performance in the match. Additionally, the goal of developing a useful, lightweight desktop application for users on all kinds of computers discourages the use of heavy machine learning libraries or massive cloud servers. Instead, the aim was to extract heavier aspects of the algorithm into a purely development-only phase, storing their outputs with the program, and then, when analysing a user's own data generated as they use the app on their own computer, simply using pure maths, lightweight `numpy` arrays and contextual heuristics. By decoupling heavy data science from the live scoring engine, the application delivers enterprise-grade analytical depth locally and instantaneously, without any external dependencies.

Before the engine can process the match data, it must confront the strict limitations of this native telemetry API. Real-world football analytics engines (such as those powered by *Opta*) benefit from hundreds of granular micro-events, including "Key Passes," "Interceptions," and "Progressive Carries." The Analytics Engine, however, is constrained by the rigid, 16-metric outfield matrix provided by the game's post-match extraction.

Because the engine lacks advanced spatial data, it cannot measure a player's creativity purely through counting stats. To bridge this gap, the algorithm synthesises a 17th metric - Expected Threat (xT) - derived from a combination of passing, dribbling, and physical exertion parameters. This final 17-element array serves as the absolute boundary of the engine's worldview. Everything the algorithm understands about a player's performance must be mathematically deduced from this highly restricted telemetry matrix.

Despite the existing limitations, the goal of this algorithm was to eliminate the black box commonly used by industry-standard services. The architecture takes messy OCR-extracted telemetry data and employs a transparent, purely mathematical approach to distil it into a single player rating. Similar to professional alternatives, there was a strong emphasis on context - considering positional variance, intelligent tactical roles, team performance, and match

configurations. Ultimately, this algorithm aims to synthesise these complex contextual variables into a single, transparent metric, transforming raw telemetry into a definitive and unbiased reflection of true on-pitch value.

II. Data Pre-Processing & Normalisation

Every statistical model, regardless of its complexity, is bound by one fundamental law: the quality of its output is entirely dependent on the quality of its inputs. After validating the extracted telemetry, the application verifies that the values fall within expected ranges before routing them through the algorithm and applies common-sense rules, such as goals cannot be greater than shots, and assists cannot be greater than passes. However, in its rawest form, this extracted telemetry is simply a collection of isolated event counts, inherently devoid of context. Feeding this unadjusted data directly into a complex scoring algorithm would create highly suspect outputs, as it treats all events as equal, regardless of the opportunities provided to the player.

In football, this concept of opportunity is inextricably linked to time. Evaluating a player purely on their gross accumulation of stats heavily skews the data in favour of those who complete a full match, whilst punishing those who are substituted on or off. These raw counts hide efficiency; a player registering 15 passes in 90 minutes has performed quite poorly in that area, whilst a player who has achieved the same number in a 15-minute cameo has performed quite well. To accurately judge a performance, the data must reflect a rate of action rather than a raw sum. Across football statistics, this is usually expressed as a rate per 90: the number of actions the player would have completed had he played 90 minutes. Statistics shown across a season, such as goals, are often presented this way: goals per 90. This method shows how well a player has performed relative to the opportunities they have had, accounting for injuries and adding parity to player comparisons.

When translating this logic to a video game simulation like EA FC, the temporal problem doubles. Unlike real-world football, simulation telemetry is fixed to two clocks: the 90 minutes of the football match and a customisable half-length game setting. This setting sets the number of real-world minutes each half takes, which can range from 4 to 20 minutes, and the in-game 45 minutes scales accordingly. The problem this poses is that, with more real-world time to play, and for instance press the pass button more times, the volume counts will be higher in longer half-length matches. To ensure mathematical fairness and cross-match comparability, the Analytics Engine must first pass all raw OCR-extracted data through a strict pre-processing pipeline. This phase standardises the volume metrics and masks low-volume anomalies, ensuring the foundational data is perfectly level before any scoring or positional weighting is applied.

Temporal Standardisation

To address the half-length problem, the engine introduces a baseline half-length setting that scales all cumulative volume stats, regardless of their original half-length. This baseline is set at 10 minutes, and all cumulative volume stats (passes, shots, tackles, etc.) are scaled to it. The maths to achieve this is simple, multiplying the raw stat by the ratio of the baseline half-length to the real half-length ($X_{adj} = X_{raw} \times \left(\frac{H_{base}}{H_{match}} \right)$). For example, if a user is playing 4-minute halves, then a midfielder completing 20 passes is playing quite well. However, if this were directly compared to a 10-minute half (where midfielders playing the full match usually average around 40 passes) without standardisation, it would look quite poor. To scale this volume up, the algorithm passes the metric through the standardisation formula ($20 \times \left(\frac{10}{4} \right) = 50$), and it is converted to its 10-minute half equivalent: 50 passes. This performance can now be fairly compared against a performance from a match with 10-minute halves.

Handling Substitute Volatility (Bayesian Smoothing)

While temporal standardisation fixes EA FC's custom clock, the engine must still apply the standard fix: per-90 normalisation. However, simply converting these adjusted stats into a per-90 rate introduces a new mathematical hazard: substitute volatility. If an 88th-minute substitute comes on and takes a single shot on target in two minutes of game time, this standard formula would give a per 90 rate of an absurd 45 shots per game. If the program fed that inflated rate directly into the scoring matrix, it would break the algorithm and generate a 10.0 rating for a two-minute cameo.

To prevent this, the engine demands proof of sustainability. It applies a statistical technique known as Bayesian smoothing, which acts as a mathematical anchor. By blending the player's observed stats with a historical positional average, the engine ensures that a player's rating is closely tied to the mean (generating a standard 6.0 rating) when they first step onto the pitch. The fewer minutes a player has on the field, the stronger this anchor holds, meaning their calculated output is largely dictated by historical expectations rather than their brief cameo. Therefore, if they played for only a few minutes, it is highly unlikely their rating would be far from the standard 6.0. To break away from this baseline and earn an elite (or abysmal) rating, the player must physically accrue enough minutes - and consistent statistical output - to overpower the positional average.

III. Live Feature Engineering & Standardisation

With the raw telemetry successfully cleaned, standardised, and insulated against substitute volatility, the foundational data entering the algorithm is mathematically stable. However, even after this pre-processing, these stats cannot be fed directly into a rating matrix. In their current form, they lack two critical mathematical properties: dimension and equivalence.

Football is an inherently multidimensional sport, and evaluating a player requires balancing their offensive output, defensive contributions, and possession retention. Raw counting stats offer no

way to compare these differing actions; one cannot simply add "3 tackles" to "50 passes" to deduce a match rating. Is 50 a good number of passes? And if it is, it is unlikely 50 is a similarly good number of tackles, or shots, for example.

To resolve this problem of comparing apples to oranges, the Analytics Engine executes a rigorous live feature engineering pipeline. This phase synthesises advanced metrics using mathematical estimations and proxies to replace missing spatial context. It then converts all telemetry into a universally comparable scale and enforces strict minimum-volume thresholds to ensure that a player's calculated efficiency is fully earned.

The Expected Threat (xT) Proxy

In real-world football analytics, the Expected Threat (xT) metric, like related metrics such as Expected Goals (xG), is calculated using precise X/Y pitch coordinates. Expected Threat uses these to measure how much a player's actions increase the team's probability of scoring - how much threat they generate. It does this by mapping a player moving the ball from a low-value zone (such as around the halfway line) to a high-value zone (such as in or around the opponent's penalty box). However, the data extracted from EA FC post-match stats screens entirely lacks spatial context. In the game's raw dataset, a five-yard sideways pass between two centre-backs is indistinguishable from a 40-yard line-breaking pass from a playmaker. This discrepancy is made worse by the fact that no similar metric, such as Expected Assists (xA), is collected, which would show the quality of the chances created, regardless of whether the playmaker's teammates finished them. Therefore, the algorithm would have no way to reward a playmaker who generated numerous great chances but played with a striker who could not finish them. Without an xT metric, the algorithm would inherently reward passive, stat-padding possession just as highly as dynamic, attacking football.

To bridge this spatial void, the Analytics Engine synthesises an xT proxy by combining available metrics to approximate Expected Threat. This heuristic is grounded in real-world sports science - research from the CIES Football Observatory and tracking data from UEFA Euro 2016 demonstrate that while total distance covered has little correlation to match success, high-intensity sprinting is statistically linked to positive goal differentials and is most commonly seen among attacking players.^{[1][2]} Furthermore, industry analysis shows that elite teams combine this physical intensity with high passing volume to generate a true attacking threat.^[3] By mathematically binding a player's sprint ratio to their passing efficiency, the engine accurately isolates box-to-box midfielders, overlapping fullbacks, and direct wingers who drive progressive, line-breaking possession.

Crucially, this proxy cannot scale linearly. A player increasing their passing volume from 20 to 40 accurate passes usually signifies a massive increase in active match involvement and threat. However, a player increasing from 80 to 100 passes is often just recycling safe possession in a deep defensive block. To reflect this reality, the engine applies a natural

logarithm to the passing data. Doing so ensures that the xT bonus highly rewards players for establishing a baseline of dynamic progression, but applies mathematical diminishing returns to prevent players from artificially inflating their rating through sterile, non-threatening possession.

Live Z-Score Standardisation

Even with advanced proxies like Expected Threat, the telemetry remains dimensionally fragmented. A centre-back executing five successful tackles and a striker completing five shots on target have both registered the number 5, but the tactical weight, frequency and difficulty of those actions will be quite different. To calculate a singular match rating that encompasses all metrics appropriately, each metric must be converted to a universally comparable unit of measurement.

To achieve this dimensionality reduction, the Analytics Engine applies a mathematical technique known as Z-score standardisation. Rather than evaluating the player based on raw stat volumes, the algorithm measures their performance in standard deviations from the historical positional mean ($Z = \frac{X - \mu}{\sigma}$). Conceptually, a Z-score abandons the raw data; instead of asking 'how many passes a player completed', it asks 'how mathematically exceptional was this player's passing volume compared to a typical player in their position?'. This means that players are judged relative to the average players, giving their performances some context.

However, standard Z-score mathematics are context-blind - they only measure distance from the mean. If a player commits five fouls in a match, whilst the historical average is one, the standard formula will yield a massive, positive z-score, just like if the player had scored 4 goals. To handle this, negative telemetry stats, where the higher they are, the worse the performance (fouls committed, offsides and possession lost) - are intercepted and handled differently. For these specific metrics, an inverted formula is used ($Z = \frac{\mu - X}{\sigma}$), ensuring that high volumes of negative actions generate heavily negative Z-scores.

Once all metrics have been run through this pipeline, instead of being expressed as raw numbers, they are decimal numbers grouped around 0, and how far above and below 0 they are shows how much the player over- or under-performed in that metric. This is expressed in standard deviations, so the majority of values will be within one standard deviation from the mean, and will therefore be between -1 and 1. Stronger outlying performances may reach as far as three standard deviations, and therefore generate a Z-score between -3 and 3. This now means that all metrics are in a form that is easily comparable - one can now look at these 12 numbers and say whether a player did better or worse than expected in each statistic, backed up by context.

Volume Masking

Whilst Z-score standardisation normalises the telemetry to a comparable scale, it fails to address a fatal flaw in the efficiency metrics (pass accuracy, shot accuracy, dribble success

rate, and tackle success rate). Because these metrics are percentages, they are very easily manipulated by low-volume events. If a defender, perhaps one who only played the second half, completed just two passes, and simple five-yard passes at that, they would achieve a 100% pass accuracy, despite their passing contribution being effectively meaningless, and in no way comparable to a player with 100% pass accuracy after 40 passes. Nevertheless, the Z-score algorithm would treat this 100% as a massive outlier and assign an elite, highly positive Z-score to a metric that is, in this case, useless.

To protect the integrity of the rating, the Analytics Engine enforces strict volume masks. A player must meet an absolute minimum volume threshold to 'unlock' their efficiency Z-score. The algorithm mandates the following baselines before it will trust the accompanying accuracy metric:

- **5 passes**
- **3 dribbles**
- **2 shots**
- **2 tackles**

These specific thresholds were chosen as they represent the absolute minimum statistical footprint required to prove intentionality rather than coincidence. For example, a player might accidentally log a 100% tackle success rate from a single lucky bounce, but executing two successful tackles requires deliberate defensive timing.

If a player fails to meet these thresholds, the algorithm does not assign a negative rating. Instead, it intercepts the efficiency Z-score and explicitly overrides it to 0.0. Because 0.0 represents the exact mathematical mean in a standard deviation matrix, the engine elegantly neutralises the metric. So if the threshold is not met, the accuracy metric is reset to the baseline, and they are not rated up or down based on it.

IV. Offline Weight Generation: Dimensionality Reduction

While Z-score standardisation successfully establishes dimensional equivalence across all metrics, it introduces a secondary architectural challenge: establishing relative tactical value. A standardised passing metric and a standardised tackling metric are mathematically comparable, but these metrics do not hold equal weight in determining a player's overall match contribution. Furthermore, this valuation is inherently position-dependent; the statistical footprint of an elite centre-back is fundamentally different from that of an elite striker.

To resolve this, the Analytics Engine explicitly rejects arbitrary, hand-tuned weighting. Manually assigning tactical value introduces developer bias, inadvertently forcing the engine to judge a user's players against a rigid, subjective tactical philosophy rather than objective performance. Instead, positional the engine mathematically derives these weights. The algorithm executes

this derivation entirely offline, leveraging a comprehensive historical dataset to calculate the weights outside of the live application environment. The resulting optimal weight matrices are then hardcoded directly into the production engine, ensuring the live software remains computationally lightweight, perfectly objective, and mathematically rigorous.

The Tangled Stats Problem

Now, the biggest problem that comes up when attempting to derive and assign weights is the tangled stats problem: football statistics are often inherently tangled (highly correlated) with one another. For example, if a midfielder has a high number of passes, they are more likely to have more assists and a higher Expected Threat. Moreover, if a defender attempts a high number of tackles, they are also more likely to win a high number of possessions and, therefore, usually have a lower number of possessions lost. Assigning weights to these stats independently, without accounting for their relationships, creates a severe problem of multicollinearity (double- or triple-counting the exact same underlying performance trait). The engine would reward a player multiple times for doing the same thing.

The PCA Solution

To mitigate the multicollinearity hazard while maintaining tactical realism, the offline analytics pipeline utilises a hybrid mathematical approach: **Guided Principal Component Analysis (PCA)**. If standard PCA is left entirely unguided, it blindly extracts whichever metrics hold the highest historical variance. Doing so introduces a severe architectural flaw; the algorithm could theoretically assign a massive attacking weight to a Defensive Midfielder simply because shooting variance happened to be highly volatile in the dataset.

To prevent this, the engine pairs the developer's overarching tactical mandate (the Macro-Philosophy) with the algorithm's mathematical precision (the Micro-Derivation). First, the algorithm groups the 17 metrics into distinct tactical blocks: *Attacking*, *Passing*, *Dribbling*, *Defending*, *Safety*, *Workrate*, and *Tactical Discipline*. For specialist positions, the engine applies structural multipliers to these blocks *before* the PCA runs. For example, the philosophy for a Central Defensive Midfielder (CDM) artificially inflates the variance of the *Defending* and *Passing* blocks, while heavily suppressing *Attacking*. (The standard Central Midfielder operates as the engine's neutral baseline, where all blocks are weighted equally).

Each component represents a naturally occurring tactical profile defined by its own unique set of internal weights. Component 1 always captures the largest chunk of variance (e.g., 45%) and maps to the dominant trait of that position, while Component 2 captures the next largest chunk (e.g., 20%) for secondary traits. Because these components are mathematically orthogonal - meaning they have zero statistical correlation with one another - the algorithm successfully isolates a player's offensive output from their defensive output.

Finally, the engine must collapse these independent dimensions back into a single, usable set of multipliers. To generate the final positional weight for a specific stat, the algorithm looks at that stat's internal weight across all components. It scales each of those internal weights strictly by the percentage of overall variance that the specific component explains, and then sums them. This method ensures the final Positional Weight Vector accurately implements the developer's tactical philosophy without double-counting the underlying telemetry.

V. The Scoring Algorithm

With the live match telemetry mathematically standardised into contextual Z-scores (Section III) and the ideal tactical philosophies distilled into independent Positional Weight Vectors (Section IV), the Analytics Engine reaches its core synthesis phase.

At this stage, the live application dynamically queries the hardcoded PCA weight matrices and aligns them with the player's live performance data. By calculating the mathematical dot product of these two arrays - multiplying every individual standardised metric by its mathematically derived tactical weight, and summing the result—the engine compresses 17 disparate dimensions of telemetry into a singular, unbounded "Raw Performance Score."

However, an unbounded raw score (e.g., +4.82 or -1.75) is unreadable to an end user who requires the traditional 1.0 to 10.0 football rating scale. Furthermore, a purely statistical dot product lacks the macro-context of the match result. The engine cannot award a defender who executes flawless tackles a perfect rating if the team simultaneously concedes four goals.

To convert this raw synthesis into a mathematically bounded, contextually aware final rating, the engine passes the raw score through a rigorous three-step pipeline: positional context modification, logistic (sigmoid) transformation, and macro-supremacy scaling.

Positional Modifiers & Contextual Heuristics

The foundational step of the scoring algorithm is the dot product: multiplying the 17 standardised Z-scores by their respective positional weights to generate a base score. However, while a dot product is perfectly linear, football is inherently non-linear and highly contextual. A purely statistical synthesis cannot account for the asymmetrical responsibilities of different tactical roles, nor can it comprehend the macro-state of the match. To bridge the gap between pure mathematics and football reality, the raw score is passed through a rigid pipeline of positional modifiers.

This modification pipeline executes four distinct architectural layers:

1. **Asymmetric Flooring:** Standardised Z-scores mathematically punish players for inaction. The engine intercepts this by applying positional floors. For example, a Centre Back is granted an "Attacking Floor" that neutralises negative Z-scores for a lack of shots or assists,

effectively treating their offensive absence as a neutral baseline (0.0) rather than a mathematical detriment.

2. **Efficiency Safeguards:** Because percentage-based stats (like pass accuracy) can generate massive negative Z-scores from tiny sample sizes (e.g., completing 1 of 3 passes), the engine caps negative efficiency variance. A poor success rate in isolated actions acts as a standard penalty rather than creating an unrecoverable mathematical black hole.
3. **Absolute Output Multipliers:** Z-score standardisation is designed to smooth out variance, but goals and assists are absolute, match-defining events. Therefore, these events are applied as flat numerical bonuses *after* the dot product calculation, ensuring that direct goal contributions always yield a massive rating spike regardless of the underlying statistical distribution.
4. **Contextual Heuristics:** The engine deploys complex, position-specific logic to evaluate how a player accumulated their stats. It does not just measure volume; it measures efficiency ratios and match context. For example, the algorithm calculates whether a striker is acting as an offensive "black hole" by comparing their turnover rate to their positive involvements, or dynamically scales a defender's clean sheet bonus based on the quality of chances (xG) the opponent created.

Logistic Mapping

After the positional modifiers are applied, the engine finally has a single raw score for the player's performance. However, because this score is based on standard deviations, it looks like a bizarre decimal (e.g., a terrible game might be -2.5 , an average game 0.0 , and a world-class game 4.8). This format is useless to an end-user. The application must convert this value into the classic 1.0 to 10.0 match rating scale used by *FIFA*, *WhoScored*, and football fans worldwide.

A naïve approach would be to scale the numbers evenly - mapping the highest historical score to a 10 and the lowest to a 1 . However, football throws up crazy outliers. If a player somehow scores 6 goals in a single game, an even scale would permanently warp the system. That 6-goal game would become the new 10.0 , and every normal, excellent performance would be crushed down to a 4.0 .

To safely lock the ratings between 1 and 10 without breaking the game, the Analytics Engine uses an "S-Curve" (mathematically known as a Sigmoid transformation).

Instead of stretching the numbers evenly, the curve is anchored to a psychological baseline: an exactly average statistical performance is locked to a **6.0 rating**. In the middle of the curve - where 95% of normal football matches happen - the scoring is highly sensitive, easily separating a decent 6.5 performance from a great 7.5 . However, at the extreme top end, the curve applies "diminishing returns." A world-class hat-trick might earn a 9.5 , and adding a fourth

goal might only push it to a 9.8. This ensures that no matter how unbelievable a player's stats become, the rating gracefully slows as it approaches 10.0, never breaking the ceiling.

VI. Macro-Context, Hybridisation & Goalkeepers

While the logistic curve successfully synthesises individual telemetry into a recognisable 1-to-10 rating, evaluating a player purely in a statistical vacuum is inherently flawed. Football is not a static environment. The match's overall momentum deeply influences a player's individual metrics; their tactical role can change depending on the phase of play, and specific positions operate under entirely different physical and data rules.

To finalise the match rating, the Analytics Engine applies three macro-resolutions. It introduces a team-level environment scalar, deploys a multi-positional hybridisation formula for players who shift roles mid-match, and completely isolates Goalkeepers into their own bespoke analytical pipeline.

The Match Supremacy Scalar

In football, it is significantly easier to string together 50 passes and register an assist when your team has 70% possession and is pinning the opponent in their own box. Conversely, a centre-back on a team facing an absolute onslaught will naturally accumulate massive tackling and clearance numbers simply because the ball is always near them.

Statistical research into football telemetry has proven that evaluating a player in a vacuum is flawed. In fact, roughly 26% of a player's statistical variance is dictated entirely by how dominant their team was on the day^[4]. If an algorithm ignores this, it will artificially inflate the ratings of players on dominant teams (stat-padding) while systematically punishing players on struggling teams.

To neutralise this environmental bias, the Analytics Engine applies a final macro-adjustment to the 1-to-10 score: **The Match Supremacy Scalar**.

The engine calculates which team genuinely dominated the match by comparing the two teams' Expected Goals (xG). Based on this ratio, the engine applies a bidirectional adjustment:

- **The Inflation Penalty:** If a player's team heavily dominated the match, the engine applies a slight penalty to their final rating. This strips away the "easy game" padding and ensures their 9.0+ rating was actually earned through individual brilliance, not just team superiority. (To prevent unfairly punishing elite teams, this penalty is strictly capped).
- **The Siege Bonus:** If a player's team was heavily outplayed, the engine applies a rating boost. This rewards players who maintained a solid baseline output despite being tactically besieged, recognising their performance against overwhelming odds.

Multi-Positional Hybridisation

Modern football is increasingly positionless. Tactical fluidity often dictates that players change roles mid-match - a fullback may invert into central midfield, or a winger may transition into a second striker, depending on the game state.

However, evaluating these hybrid performances introduces a major telemetry constraint. When a user enters multiple positions for a player (e.g., "CB, RB" in the UI), the Analytics Engine only has aggregated match stats. The data is "blind" - the engine does not know if the player spent 80 minutes at CB and 10 minutes at RB, nor does it know which tackles occurred in which role.

To resolve this, the engine embraces parallel evaluation. It uses the player's entire unpartitioned match data and runs it through the independent weight matrices for each entered position, generating multiple distinct match ratings.

Collapsing these multiple ratings into a single final score presents an architectural challenge:

- **The Accountability Void (Maximum):** The engine cannot simply award the highest rating and discard the rest. If a player puts in a world-class performance in their main role but performs disastrously when asked to cover a secondary role for a portion of the match, they must be held accountable for that poor stint. Ignoring the lower rating entirely grants them an unearned free pass.
- **The Versatility Punishment (Average):** Conversely, the engine cannot use a simple average. If a player earns a 9.0 in their primary role but a 6.0 in the secondary role due to unfamiliarity or lack of volume, a pure average (7.5) unjustly destroys their rating, punishing them far too heavily for stepping in to fill a tactical gap.

To bridge this gap, the engine deploys a proprietary **Alpha Drag** system. The algorithm assumes the player's highest calculated rating reflects their primary tactical role. It anchors the final score to this maximum rating, but applies a fractional "drag" based on the lower scores. This ensures the player is rightfully penalised for poor secondary performances, but the punishment is scaled down so it does not erase their primary success.

Goalkeeper Isolation Heuristics

The structural realities of the goalkeeper position render standard analytics algorithms entirely obsolete. This is not merely a tactical difference; it is a fundamental data constraint. For outfield players, the Analytics Engine receives a 17-metric telemetry matrix (measuring progression, passes, distance covered, xT, etc.). For goalkeepers, the engine receives a completely different, highly specialised 11-metric matrix consisting purely of reactive events (e.g., *shots against*, *saves*, *rush saves*, *punch saves*, *penalty metrics*).

Attempting to force an 11-metric reactive array through a 17-dimensional PCA weight matrix designed for outfielders would cause the linear algebra to collapse mathematically. Therefore, the engine completely isolates goalkeepers from the outfield architecture, routing them into a bespoke, dedicated heuristic pipeline.

The core of this pipeline is **Expected Goals Prevented (xGP)**. Rather than simply counting raw saves, the engine calculates the opponent's total Expected Goals and subtracts the actual goals conceded. This creates a true, context-aware measurement of shot-stopping value.

Furthermore, the goalkeeper pipeline deliberately reverses several contextual heuristics used for outfielders to reflect their reactive role. The most prominent example is the "Inverted Clean Sheet Bonus." When a central defender keeps a clean sheet against a team that generated 0.5 xG, the defender receives a massive rating boost because they actively smothered the opponent. However, if a goalkeeper keeps a clean sheet against 0.5 xG, it indicates they were largely a "passenger" in a game dominated by their own defence, yielding a minimised bonus. Conversely, if the defence collapses and allows 2.5 xG, but the goalkeeper still secures a clean sheet, they trigger a "Bailout" bonus and receive a massive rating spike for single-handedly carrying the team.

VII. Conclusion

A tension between statistical objectivity and tactical reality has historically hampered the quantification of football performance. Commercial rating systems and native in-game algorithms often rely on rigid, one-size-fits-all linear models. These models fail to account for asymmetric responsibilities across positions, aggressively punish players for executing specific managerial instructions, and unquestioningly reward players for hierarchical dominance within their team.

The custom Analytics Engine detailed in this whitepaper was engineered to resolve these epistemological flaws. By standardising raw telemetry into variance-aware Z-scores (Section III), the engine creates a mathematically levelled playing field across all metrics. By applying the Minimum Covariance Determinant and Eigen-Decomposition (Section IV), the algorithm actively hunts for the true tactical DNA of every position, filtering out statistical noise and deriving perfect Positional Weight Vectors without human bias.

Most importantly, the engine wraps this linear algebra in rigorous, context-aware heuristics (Sections V & VI). By integrating synthetic xG models, the Alpha Drag hybridisation coefficient, and the GLMM-backed Match Supremacy Scalar, the algorithm bridges the gap between a spreadsheet and the pitch.

The resulting output is a robust, dynamic 1.0 to 10.0 rating system that does not merely count what a player did, but mathematically evaluates *how* and *why* they did it. It successfully mimics

the subjective, nuanced eye of an elite football analyst while maintaining the strict, reproducible objectivity of a data science framework.

Footnotes

1. Drs Raffaele Poli, Loïc Ravenel and Roger Besson (2021). Monthly Report 68: Analysis of distances covered in professional football. *CIES Football Observatory* [Available Here](#) ↩
2. Kubayi A. (2019). Evaluation of match-running distances covered by soccer players during the UEFA EURO 2016. *South African journal of sports medicine*, 31(1), v31i1a6127. [Available Here](#) ↩
3. Matthew Herring-Rogers. Distance in football matters: Use it wisely. *PlayerData*. [Available Here](#) ↩
4. James Ball, Minh Huynh and Matthew C. Varley (2025). Comparing player rating systems as a metric for assessing individual performance in soccer. *Journal of Sports Sciences* [Available Here](#) ↩